

In **Nmap**, the `--disable-arp-ping` option forces the scanner to ignore its default behavior of using **Address Resolution Protocol (ARP)** for host discovery on local Ethernet networks. 

Why use it?

- **Avoid Detection:** On a local network, Nmap typically uses ARP because it is faster and more reliable than ICMP ("ping"). Disabling it allows you to test how a host responds to other discovery methods like **ICMP** or **TCP**.
- **Bypass Spoofing:** It is essential when a device is spoofed with a false IP address. **ARP or ARP spoofing**, which can provide false IP address for every IP address is active.
- **Simulate External Scanning:** By disabling ARP, you can simulate external scanning from a remote or public network, allowing you to test how a host responds to specifically target non-ARP traffic.

Key Technical Details

- Default Behavior:** If Nmap detects the target is on the network, it will send a ping before the scan. This is the default even if you specify other ping types like `-PE` (ICMP Echo).
- Alternative Flag:** The `--send-ip` flag can achieve a similar result, as it only sends raw IP packets. This is useful to send packets at the IP layer rather than the raw Ethernet layer, which is not supported on all platforms (e.g., Windows).
- Combination with `-Pn`:** Even if you use `-Pn` (skip host discovery), Nmap may still perform an ARP scan to determine the target's MAC address for further port scanning unless this flag is used.

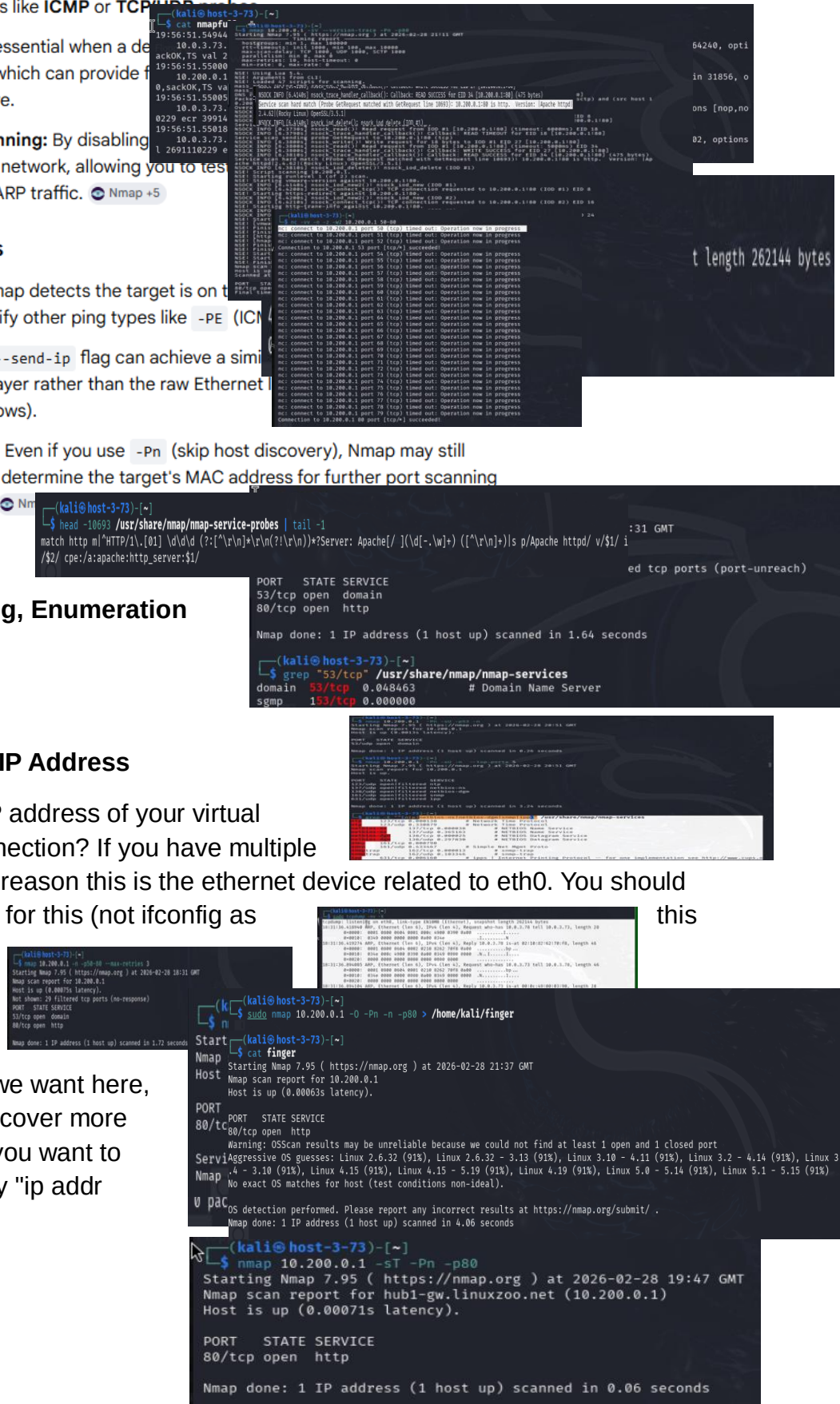
Lab 3

Active Recon -

Network Scanning, Enumeration

Question 1 Network IP Address

What is the primary IP address of your virtual machine network connection? If you have multiple connections for some reason this is the ethernet device related to eth0. You should use the "ip" command for this (not ifconfig as this is very old and weak). You can use "ip help" to discover the options. The "addr" option is what we want here, and again you can discover more with "ip addr help". If you want to jump to the answer, try "ip addr



show".

```
(kali@host-3-73)-[~]
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:49:00:03:90 brd ff:ff:ff:ff:ff:ff
   inet 10.0.3.73/29 brd 10.0.3.79 scope global dynamic noprefixroute eth0
       valid_lft 859491sec preferred_lft 859491sec
   inet6 fe80::20c:49ff:fe00:390/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: labbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc htb state DOWN group default qlen 1000
   link/ether 52:54:00:a6:b7:32 brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.254/24 brd 192.168.1.255 scope global labbr0
       valid_lft forever preferred_lft forever
```

Machine ip : **10.0.3.73**

Check Tests: Ready

Network	PAS
Address	S

Question 2 Mac address

This time use "ip link help" and "ip link show" and find out the mac address of eth0. Enter it in the format 00:00:00:00:00:00 using lowercase.

```
$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
   link/ether 00:0c:49:00:03:90 brd ff:ff:ff:ff:ff:ff
3: labbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc htb state DOWN mode DEFAULT group default qlen 1000
   link/ether 52:54:00:a6:b7:32 brd ff:ff:ff:ff:ff:ff
```

Machine mac: **00:0c:49:00:03:90**

Check Tests: Ready

Layer 2	PAS
Address	S

Question 3 Network Gateway

This time use "ip route help" and "ip route show" and find out the IP address of your machine's gateway. The gateway is where your packets would be sent if it is destined for an IP address which is not in the local subnet

```
(kali@host-3-73)-[~]
$ ip route show
default via 10.0.3.78 dev eth0 proto dhcp src 10.0.3.73 metric 100
10.0.3.72/29 dev eth0 proto kernel scope link src 10.0.3.73 metric 100
192.168.1.0/24 dev labbr0 proto kernel scope link src 192.168.1.254 linkdown
```

Machine gateway: **10.0.3.78**

Check Tests: Ready

Gateway PAS
IP S

Question 4 Gateway Mac

Look into /proc/net/arp. What is the mac address of the gateway? Enter it in the format 00:00:00:00:00:00 using lowercase.

Gateway mac: **e2:b5:36:6a:c3:e9**

Check Tests: Ready

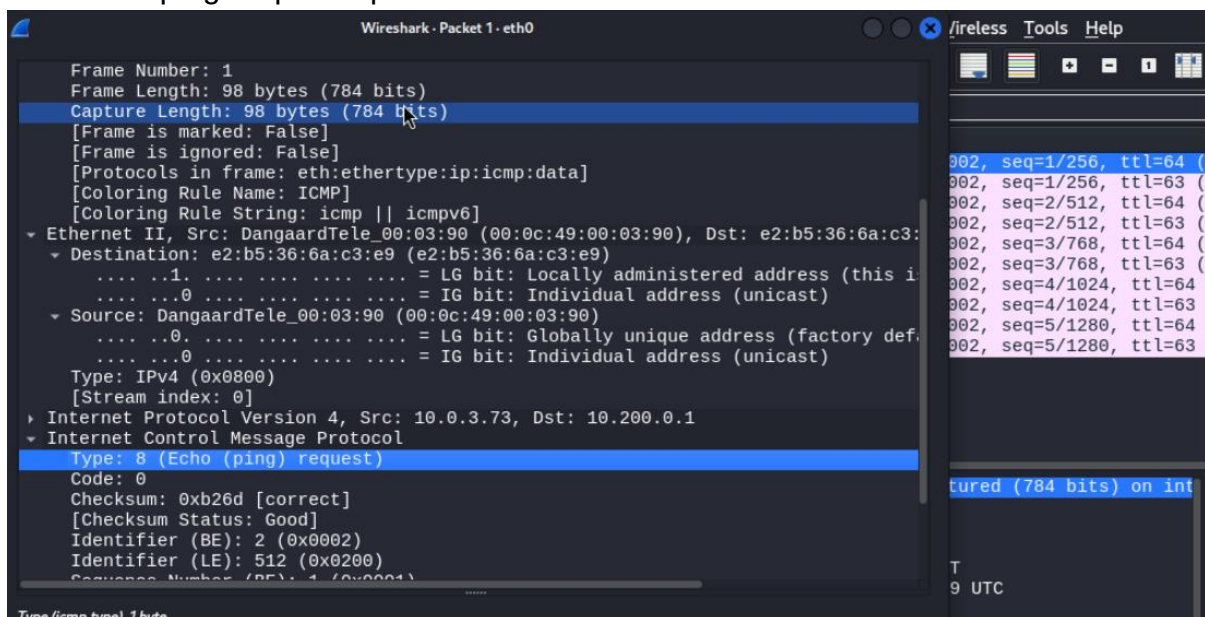
Gateway PAS
mac S

Question 5 Monitoring Scanning Traffic - Wireshark and tcpdump

Run Wireshark in Kali. It can be found in the graphical menu.

9. Sniffing & Spoofing -> Wireshark

Start a live capture, and in a kali terminal "ping" the machine at 10.200.0.1 with 5 pings (you can use the "-c" option of ping for this, or press CTRL C to stop the ping). This ping should send a number of ping request packets, and if the network is operating correctly and the a target is up, you should receive a corresponding number of ping response packets.



What does wireshark report:**98**

is the length of a ping packet:**8**

is the ICMP type number of a ping request packet:

Check Tests: Ready

Length in bytes	PAS S
Type number for request	PAS S

Now, open another terminal window, and use tcpdump to check its options using -h. Which tcpdump flag:

```
(kali@host-3-73)~$ tcpdump -h
tcpdump version 4.99.5
libpcap version 1.10.5 (with TPACKET_V3)
OpenSSL 3.3.2 3 Sep 2024
64-bit build, 64-bit time_t
Usage: tcpdump [-AbdDefhHlJKLLnNOpqStuUvxxX#] [-B size] [-c count] [--count]
[-C file_size] [-E algo:secret] [-F file] [-G seconds]
[-i interface] [--immediate-mode] [-j timestamp]
[-M secret] [--number] [--print] [-Q in|out|inout]
[-r file] [-s snaplen] [-T type] [--version]
[-V file] [-w file] [-W filecount] [-y datalinktype]
[--time-stamp-precision precision] [--micro] [--nano]
[-z postrotate-command] [-Z user] [expression]
```

```
(kali@host-3-73)~$ tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.labbr0 [Up, Disconnected]
5.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
6.nflog (Linux netfilter log (NFLOG) interface) [none]
7.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
8.dbus-system (D-Bus system bus) [none]
9.dbus-session (D-Bus session bus) [none]
```

is used to display the list of network interfaces: **-D**

is used to display numerical port numbers and not perform DNS host resolution: **-n**

Check Tests: Ready

Display Interfaces flag	PAS S
Display Numeric Output flag	PAS S

Now start a tcpdump capture on eth0, using numerical output for hosts and ports and displaying the contents of sniffed packets in Hex and ASCII. In your other kali terminal use ping again to send just a few packets to 10.200.0.1. It may not be obvious where the ICMP type is in the tcp dump output. Try clicking on it in Wireshark, and comparing the data pane (pane 3 in Wireshark) with the tcpdump Hex and ASCII output!

As tcpdump on eth0 requires administrator privileges, run it using sudo tcpdump.

What does the tcpdump output show:

```
(kali@host-3-73)-[~]
$ ping -c 3 10.200.0.1
PING 10.200.0.1 (10.200.0.1) 56(84) bytes of data.
64 bytes from 10.200.0.1: icmp_seq=1 ttl=63 time=0.759 ms
64 bytes from 10.200.0.1: icmp_seq=2 ttl=63 time=0.727 ms
64 bytes from 10.200.0.1: icmp_seq=3 ttl=63 time=0.670 ms

— 10.200.0.1 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2022ms
rtt min/avg/max/mdev = 0.670/0.718/0.759/0.036 ms
```

What is the protocol of the ping packets: **ICMP**

What are the last 8 characters in the content of each ICMP packet's data: **01234567**

```
(kali@host-3-73)-[~]
$ sudo tcpdump -i eth0 -n -X icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:58:33.412176 IP 10.0.3.73 > 10.200.0.1: ICMP echo request, id 5, seq 1, length 64
    0x0000: 4500 0054 da03 4000 4001 4894 0a00 0349  E..T..@.@.H...I
    0x0010: 0ac8 0001 0800 778a 0005 0001 59e9 a169  ....w....Y..i
    0x0020: 0000 0000 c049 0600 0000 0000 1011 1213  ....I.....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223  .....!"#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233  $%&'()*+,-./0123
    0x0050: 3435 3637                                4567
18:58:33.412855 IP 10.200.0.1 > 10.0.3.73: ICMP echo reply, id 5, seq 1, length 64
    0x0000: 4500 0054 7003 0000 3f01 f394 0ac8 0001  E..Tp...?.....
    0x0010: 0a00 0349 0000 7f8a 0005 0001 59e9 a169  ...I.....Y..i
    0x0020: 0000 0000 c049 0600 0000 0000 1011 1213  ....I.....
    0x0030: 1415 1617 1819 1a1b 1c1d 1e1f 2021 2223  .....!"#
    0x0040: 2425 2627 2829 2a2b 2c2d 2e2f 3031 3233  $%&'()*+,-./0123
    0x0050: 3435 3637                                4567
```

Check Tests: Ready

tcpdump default protocol PAS

S

tcpdump default protocol PAS

S

Question 6 Network Sweeping for Hosts with ping

In this set of questions you will learn how to script the ping tool to for a range of IP addresses. This is particularly useful when you are using a network command which normally only targets a single machine but you want to target a range of IP numbers, such as with network sweeping.

Firstly, ping 10.200.0.1 using "-c 1" (so only send a single packet exchange). What is the TTL of the response to this ping packet?

```
(kali@host-3-73)-[~]
$ ping -c 1 10.200.0.1
PING 10.200.0.1 (10.200.0.1) 56(84) bytes of data.
64 bytes from 10.200.0.1: icmp_seq=1 ttl=63 time=0.850 ms

--- 10.200.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.850/0.850/0.850/0.000 ms

(kali@host-3-73)-[~]
$
```

```
...0 0000 0000 0000 = Fragment Offset: 0
Time to Live: 63
Protocol: ICMP (1)
Header Checksum: 0x565e [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.200.0.1
Destination Address: 10.0.3.73
[Stream index: 0]
Internet Control Message Protocol
```

TTL: 63

Check Tests: Ready

TTL	PAS
Correct	S

It is possible to use bash variables in commands. One way is to set a variable using a for loop construct. Look at the following example:

```
for i in {1..10}; do mkdir /home/kali/newdir$i; done;
```

The "for" part till the first ";" sets up a loop which sets a variable called "i" to the values 1,2,3,4,5,6,7,8,9,10, one after another. For each value the "do" command is executed; "done" indicates the end of the loop. The \$i is replaced by the current value of i. In this example, "newdir\$i" becomes "newdir1", "newdir2", etc, up to "newdir10".

```
(kali@host-3-73)-[~]
$ for i in {1..10}; do echo 10.200.0.$i; done
10.200.0.
10.200.0.
10.200.0.
10.200.0.
10.200.0.
10.200.0.
10.200.0.
10.200.0.
10.200.0.
10.200.0.

(kali@host-3-73)-[~]
$
```

```
(kali@host-3-73)-[~]
$ sudo tcpdump -vn -i eth0 icmp > /home/kali/dump1
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

Consider another example:

```
for i in {1..10}; do echo 10.200.0.$i; done;
```

How many IP numbers are output? Count: **10**

Check Tests: Ready

Count Correct PAS
S

In this question you will use a bash for loop and ping to investigate automation of network scanning for hosts. Wireshark is excellent for packet analysis, but sometimes it can be resource hungry, so this time use tcpdump to capture a ping sweep. The tcpdump tool allows us to capture network traffic on a particular network device from the cmd line.

Make sure you have two terminal windows open. In one window, which will deal with the monitoring of the network, we will use tcpdump to sniff for ICMP traffic and redirect the output to a file:

```
tcpdump -vn -ieth0 icmp > /home/kali/dump1
```

This will start the capture. Press CTRL C in this window when you want the capture to end. These flags indicate that it is a verbose capture (-v), and that IP numbers will be kept numeric and not generate DNS lookups (-n), and the capture is on interface eth0 (-i eth0). The captured information is saved to the dump1 file.

In the other terminal window, which in turn will generate the packets during the experiment, do the following REPLACING ?????? with suitable parameters to make the loop go from 1 to 30, and for the ping to send packets to 10.200.0.1 up to 10.200.0.30, depending on i.

```
for i in {?????}; do ping -W1 -c1 ?????; done;
```

This will run a network sweep of IPs in the range 10.200.0.1 to 10.200.0.30. It is not fast... may take around a minute. The -W sets the timeout to 1 second each time, just to make the question go a bit faster. Note though in the real world a 1 second

timeout might mean you miss things...

```
kali@host-3-73:~$ for i in {1..30}; do ping -W1 -c1 10.200.0.$i; done
PING 10.200.0.1 (10.200.0.1) 56(84) bytes of data.
64 bytes from 10.200.0.1: icmp_seq=1 ttl=63 time=0.799 ms

--- 10.200.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.799/0.799/0.799/0.000 ms
PING 10.200.0.2 (10.200.0.2) 56(84) bytes of data.

--- 10.200.0.2 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

PING 10.200.0.3 (10.200.0.3) 56(84) bytes of data.

--- 10.200.0.3 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

PING 10.200.0.4 (10.200.0.4) 56(84) bytes of data.
64 bytes from 10.200.0.4: icmp_seq=1 ttl=64 time=0.527 ms

--- 10.200.0.4 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.527/0.527/0.527/0.000 ms
PING 10.200.0.5 (10.200.0.5) 56(84) bytes of data.

--- 10.200.0.5 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms

PING 10.200.0.6 (10.200.0.6) 56(84) bytes of data.
```

How many live hosts has the ping sweep found: **2**

Once complete stop the tcpdump capture.

Check Tests: Ready	
Capture seems to start	PAS S
Capture seems to end	PAS S
Live hosts found	PAS S

Use cat or less to examine the packet capture, and locate the ICMP echo reply packets. The ICMP packet is contained within an IP packet, and the related IP header information is on the line just before the ICMP information. From that

information, discover the IP packet id number for the last received echo reply.

```
Internet Protocol Version 4, Src: 10.200.0.4, Dst: 10.0.3.73
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    0000 00.. = Differentiated Services Codepoint: Default (0)
    .... ..00 = Explicit Congestion Notification: Not ECN-Capable Transport (0)
  Total Length: 84
  Identification: 0x5962 (22882)
  000. .... = Flags: 0x0
    0... .... = Reserved bit: Not set
    .0.. .... = Don't fragment: Not set
    ..0. .... = More fragments: Not set
```

ICMP Packet - IP Header Id number: **22882**

Check Tests: Ready	
dump1 seems ok test1	PAS S
dump1 seems ok test2	PAS S
Last ICMP reply packet - IP header id no	PAS S

Use grep and wc. How many echo requests were sent?

```
(kali@host-3-73)-[~]
$ ls
Desktop  Downloads  Music  Number  Public  Templates
Documents dump1      num    Pictures request  Videos

(kali@host-3-73)-[~]
$ grep "echo request" dump1 | wc -l
30
```

Number of echo requests: **30**

Check Tests: Ready	
dump1 seems ok test1	PAS S
dump1 seems ok test2	PAS S
Echo requests	PAS S

Use grep, looking for packets with "echo reply", and use the '-B1' to also show the line immediately before the line which matches (which contains the packet header

information of the reply). Sometimes the ttl is 64, but sometimes the ttl is something else.

What is the other ttl: **63**

Why? The response was sent slower in this case The response was changed by a firewall The response was specially engineered The response came from a machine with a different default ttl The ttl is smaller as the machine in question is nearer in terms of network hops The ttl is smaller as the machine in question is further away in terms of network hops Switches use one ttl, routers use another Compression was used in one case The TTL only indicates the number of milliseconds it takes to process the icmp packet

Check Tests: Ready		
dump1 seems ok	PAS	
test1	S	
dump1 seems ok	PAS	
test2	S	
Other ttl	PAS	
	S	
reason	PAS	
	S	

Sometimes it's useful to be able to filter out traffic we aren't interested in. Use tcpdump again, but this time capture only traffic which is not LinuxZoo remote administration traffic. So, ignore ssh and telnet traffic using a filter.

tcpdump -ieth0 -vn not port ssh and not port telnet

Direct that file to save the capture to /home/kali/dump2, and terminate the capture after capturing some host discovery packets, using arping as follows.

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -vn not port ssh and not port telnet > /home/kali/dump2
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C22 packets captured
22 packets received by filter
0 packets dropped by kernel
```

```
(kali@host-3-73)-[~]
$ sudo arping -ieth0 -c2 10.0.3.73
ARPING 10.0.3.73
Timeout
Timeout

— 10.0.3.73 statistics —
2 packets transmitted, 0 packets received, 100% unanswered (0 extra)

(kali@host-3-73)-[~]
$
```

We can use the arping tool to check if a host on a local network is up. Try: "arping -ieth0 -c2 10.200.0.1", replacing the IP "10.200.0.1" with the IP of your virtual machine gateway address.

Check Tests: Ready

dump2 seems ok	PAS
test1	S
dump2 seems ok	PAS
test2	S

What is the protocol arping has used to sent the host discovery packets?
arping protocol (use upper case): **ARP**

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -vn not port ssh and not port telnet > /home/kali/dump2
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C10 packets captured
10 packets received by filter
0 packets dropped by kernel
```

What is the packet length of the reply to the arping from your gateway?

packet length: 48

```
l-$ sudo tcpdump -ieth0 -vn not port ssh and not port telnet > /home/kali/dump2
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C10 packets captured
10 packets received by filter
0 packets dropped by kernel

(kali@host-3-73)-[~]
$ grep "ARP, Reply" dump2

(kali@host-3-73)-[~]
$ cat dump2
20:37:00.872098 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.73 tell 10.0.3.78, length 46
20:37:00.872158 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.73 is-at 00:0c:49:00:03:90, length 28
20:37:02.162679 IP (tos 0x0, ttl 64, id 9405, offset 0, flags [DF], proto UDP (17), length 68)
  10.0.3.73.42962 > 10.200.0.1.53: 33099+ A? host-3-73.linuxzoo.net. (40)
20:37:02.162971 IP (tos 0x0, ttl 64, id 9406, offset 0, flags [DF], proto UDP (17), length 68)
  10.0.3.73.42962 > 10.200.0.1.53: 40272+ AAAA? host-3-73.linuxzoo.net. (40)
20:37:02.163997 IP (tos 0x0, ttl 63, id 27070, offset 0, flags [none], proto UDP (17), length 84)
  10.200.0.1.53 > 10.0.3.73.42962: 33099* 1/0/0 host-3-73.linuxzoo.net. A 10.0.3.73 (56)
20:37:02.163998 IP (tos 0x0, ttl 63, id 27071, offset 0, flags [none], proto UDP (17), length 109)
  10.200.0.1.53 > 10.0.3.73.42962: 40272* 0/1/0 (81)
20:37:02.231592 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.78 tell 10.0.3.73, length 44
20:37:02.231977 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.78 is-at e2:b5:36:6a:c3:e9, length 46
20:37:03.233429 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.78 tell 10.0.3.73, length 44
20:37:03.233802 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.78 is-at e2:b5:36:6a:c3:e9, length 46
```

Check Tests: Ready

dump2 seems ok	PAS
test1	S
dump2 seems ok	PAS
test2	S
protocol	PAS
	S
dump2 byte len of is-	PAS
at	S

Question 7 Host sweeping with arping

Use the "ip route show" command and identify the start and end address (not including the network number and broadcast address) of your virtual machine's network. Use the network number and netmask to make this calculation from the "ip route show" command output.

```
(kali@host-3-73)-[~]
$ ip route show
default via 10.0.3.78 dev eth0 proto dhcp src 10.0.3.73 metric 100
10.0.3.72/29 dev eth0 proto kernel scope link src 10.0.3.73 metric 100
192.168.1.0/24 dev labbr0 proto kernel scope link src 192.168.1.254 linkdown

(kali@host-3-73)-[~]
$
```

Start IP: **10.0.3.73**

End IP: **10.0.3.78**

Check Tests: Ready

start ip is PAS
right S

end ip is right PAS
S

```
#!/bin/bash
```

```
for i in {0..150}; do
```

```
    arping -c 2 192.168.100.$i | grep 'bytes from' | awk '{print " possible target up  
at: " $4 " " $5}' | sort -u
```

```
done
```

Consider the above bash host discovery script. Use an editor and write this into a script /home/kali/arpbash. After writing the script make sure the file is executable using "chmod +x /home/kali/arpbash". This script should do an arping scan of the local network, if configured correctly!

```
(kali@host-3-73)-[~]  
$ nano arpbash  
  
(kali@host-3-73)-[~]  
$ ls  
Desktop Downloads dump2 num Pictures request Videos  
Documents dump1 Music Number Public Templates  
  
(kali@host-3-73)-[~]  
$ nano arpbash  
  
(kali@host-3-73)-[~]  
$ nano arpbash  
  
(kali@host-3-73)-[~]  
$ chmod +x /home/kali/arpbash  
  
(kali@host-3-73)-[~]  
$ sudo ./arpbash > dump3  
  
(kali@host-3-73)-[~]  
$ cat dump3  
possible target up at: e2:b5:36:6a:c3:e9 (10.0.3.78):
```

In the "for" loop, which currently pings 0 to 150 IPs, change this to be the last octet of the start and end ip you identified in the previous question so we can find all the hosts on our network. Where the script has "192.168.100." change this to the first three octets of your network IP from the previous question.

Check Tests: Ready

for ok	PAS S
subnet ok	PAS S

Run the command using "sudo ./arpbash", but save the output to dump3 in /home/kali.

```
(kali@host-3-73)-[~]  
$ sudo ./arpbash > dump3  
  
(kali@host-3-73)-[~]  
$ cat dump3  
possible target up at: e2:b5:36:6a:c3:e9 (10.0.3.78):
```

Look at the file and identify the first IP number it has found.

IP: **10.0.3.78**

Check Tests: Ready

File looks	PAS
sensible	S
ip right	PAS
	S

Question 8 Network Tracing with traceroute

Gathering information on a target network topology, including network devices and other infrastructure, could be useful during a test. Attackers may also check Internet connectivity from machines inside an organisations network, such as to test if communications to a Cmd and Ctrl Server could be established.

In order for your virtual machine to reach the internet, it's packets travels through a number of virtual networks. The final network gateway node is 10.200.0.1.

Using traceroute, investigate the route to reach 10.200.0.1. Try traceroute using its default settings with traceroute -n 10.200.0.1, while in another terminal window monitoring the traffic using tcpdump, including the ASCII packet content.

What is the protocol traceroute has used to sent the network tracing packets?

traceroute protocol: **UDP**

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -nn -vv -X host 10.200.0.1
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:04:57.975119 IP (tos 0x0, ttl 1, id 62363, offset 0, flags [none], proto UDP (17), length 60)
 10.0.3.73.38522 > 10.200.0.1.33434: [bad udp cksum 0x184b → 0xd972!] UDP, length 32
  0x0000: 4500 003c f39b 0000 0111 ae04 0a00 0349  E...<.....I
  0x0010: 0ac8 0001 967a 829a 0028 184b 4041 4243  ....z...(.K@ABC
  0x0020: 4445 4647 4849 4a4b 4c4d 4e4f 5051 5253  DEFGHIJKLMNOPQRS
  0x0030: 5455 5657 5859 5a5b 5c5d 5e5f  TUVWXYZ[\]^_
21:04:57.975422 IP (tos 0x0, ttl 1, id 53066, offset 0, flags [none], proto UDP (17), length 60)
 10.0.3.73.57399 > 10.200.0.1.33435: [bad udp cksum 0x184b → 0x8fb4!] UDP, length 32
  0x0000: 4500 003c cf4a 0000 0111 d255 0a00 0349  E...<.J.....U...I
  0x0010: 0ac8 0001 e037 829b 0028 184b 4041 4243  ....7...(.K@ABC
  0x0020: 4445 4647 4849 4a4b 4c4d 4e4f 5051 5253  DEFGHIJKLMNOPQRS
  0x0030: 5455 5657 5859 5a5b 5c5d 5e5f  TUVWXYZ[\]^_
```

In the contents of the packets what are the first 4 characters after the @ symbol?

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -nn -vv -X host 10.200.0.1
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:04:57.975119 IP (tos 0x0, ttl 1, id 62363, offset 0, flags [none], proto UDP (17), length 60)
 10.0.3.73.38522 > 10.200.0.1.33434: [bad udp cksum 0x184b → 0xd972!] UDP, length 32
  0x0000: 4500 003c f39b 0000 0111 ae04 0a00 0349  E...<.....I
  0x0010: 0ac8 0001 967a 829a 0028 184b 4041 4243  ....z...(.K@ABC
  0x0020: 4445 4647 4849 4a4b 4c4d 4e4f 5051 5253  DEFGHIJKLMNOPQRS
  0x0030: 5455 5657 5859 5a5b 5c5d 5e5f  TUVWXYZ[\]^_
21:04:57.975422 IP (tos 0x0, ttl 1, id 53066, offset 0, flags [none], proto UDP (17), length 60)
 10.0.3.73.57399 > 10.200.0.1.33435: [bad udp cksum 0x184b → 0x8fb4!] UDP, length 32
  0x0000: 4500 003c cf4a 0000 0111 d255 0a00 0349  E...<.J.....U...I
  0x0010: 0ac8 0001 e037 829b 0028 184b 4041 4243  ....7...(.K@ABC
  0x0020: 4445 4647 4849 4a4b 4c4d 4e4f 5051 5253  DEFGHIJKLMNOPQRS
  0x0030: 5455 5657 5859 5a5b 5c5d 5e5f  TUVWXYZ[\]^_
```

traceroute packet content:**ABCD**

Now check the help (--help)/man page for traceroute, and find the flag to use ICMP packets, rather than the default. Run the trace again using ICMP and review the output.

```
(kali@host-3-73)-[~]
$ traceroute -I -n 10.200.0.1
traceroute to 10.200.0.1 (10.200.0.1), 30 hops max, 60 byte packets
 1  10.0.3.78  1.097 ms  0.626 ms  0.253 ms
 2  10.200.0.1  0.830 ms  0.721 ms *
```

Num of hops:2

```
(kali@host-3-73)-[~]
$ dig -x 10.200.0.1

; <<>> DiG 9.20.4-3-Debian <<>> -x 10.200.0.1
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 8190
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
; COOKIE: 874f6a6936a842fa0100000069a20926e19057363b27144f (good)
;; QUESTION SECTION:
;1.0.200.10.in-addr.arpa.      IN      PTR

;; ANSWER SECTION:
1.0.200.10.in-addr.arpa. 86400 IN      PTR      hub1-gw.linuxzoo.net.

;; Query time: 4 msec
;; SERVER: 10.200.0.1#53(10.200.0.1) (UDP)
;; WHEN: Fri Feb 27 21:14:14 GMT 2026
;; MSG SIZE rcvd: 114
```

Use dig to perform a reverse DNS lookup to find the Hostname of last hop: **hub1-gw.linuxzoo.net**

In the contents of the packets what are the 4 characters before the [symbol in the ICMP packet payload?

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -nn -vv -X icmp and host 10.200.0.1
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:21:36.976679 IP (tos 0x0, ttl 1, id 30949, offset 0, flags [none], proto ICMP (1), length 60)
  10.0.3.73 > 10.200.0.1: ICMP echo request, id 53, seq 1, length 40
    0x0000: 4500 003c 78e5 0000 0101 28cb 0a00 0349  E..<x.....(....I
    0x0010: 0ac8 0001 0800 8244 0035 0001 4849 4a4b  ....D.5..HIJK
    0x0020: 4c4d 4e4f 5051 5253 5455 5657 5859 5a5b  LMNOPQRSTUVWXYZ[
    0x0030: 5c5d 5e5f 6061 6263 6465 6667          \]^_`abcdefg
21:21:36.977514 IP (tos 0x0, ttl 1, id 30950, offset 0, flags [none], proto ICMP (1), length 60)
```

traceroute packet content:**WXYZ**

Check Tests: Ready	
protocol	PAS S
content	PAS S
Count the hops	PAS S
Last hop hostname	PAS S
ICMP content	PAS S

Question 9 Network Scanning with nmap - Host Discovery

The nmap network scanning tool can be used for a variety of different types of recon and info gathering. We will learn how to configure and use various different types of scans, starting with host discovery, and then port scanning.

Review the help for the nmap network scanning tool using the -h option. Check the section on host discovery. The online documentation is good also:

<https://nmap.org/book/man-host-discovery.html>

nmap -h | less

In another terminal window, run the nmap scanner, to perform only host discovery on a single host, with no DNS or protocol resolution. In a 2nd terminal window run tcpdump to monitor the network traffic generated by the scanner (going out on that interface).

nmap -sn -n 10.200.0.1

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -nn -vv
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:51:58.122514 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.78 tell 10.0.3.73, length 28
21:51:58.122877 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.78 is-at e2:b5:36:6a:c3:e9, length 46
21:51:58.320134 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.73 tell 10.0.3.78, length 46
21:51:58.320160 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.73 is-at 00:0c:49:00:03:90, length 28
21:52:14.047375 IP (tos 0x0, ttl 45, id 2047, offset 0, flags [none], proto ICMP (1), length 28)
  10.0.3.73 > 10.200.0.1: ICMP echo request, id 56015, seq 0, length 8
21:52:14.047908 IP (tos 0x0, ttl 41, id 2068, offset 0, flags [none], proto TCP (6), length 44)
  10.0.3.73.57705 > 10.200.0.1.443: Flags [S], cksum 0xab2a (correct), seq 3007920764, win 1024, options [mss 1460], length
21:52:14.048213 IP (tos 0x0, ttl 63, id 10676, offset 0, flags [none], proto ICMP (1), length 28)
  10.200.0.1 > 10.0.3.73: ICMP echo reply, id 56015, seq 0, length 8
21:52:14.048537 IP (tos 0xc0, ttl 63, id 10677, offset 0, flags [none], proto ICMP (1), length 72)
  10.200.0.1 > 10.0.3.73: ICMP 10.200.0.1 tcp port 443 unreachable, length 52
  IP (tos 0x0, ttl 40, id 2068, offset 0, flags [none], proto TCP (6), length 44)
  10.0.3.73.57705 > 10.200.0.1.443: Flags [S], cksum 0xab2a (correct), seq 3007920764, win 1024, options [mss 1460], length
21:52:14.048680 IP (tos 0x0, ttl 56, id 54971, offset 0, flags [none], proto TCP (6), length 40)
  10.0.3.73.57705 > 10.200.0.1.80: Flags [R], cksum 0xc443 (correct), seq 0, ack 3007920764, win 1024, length 0
21:52:14.049005 IP (tos 0x0, ttl 41, id 43949, offset 0, flags [none], proto ICMP (1), length 40)
  10.0.3.73 > 10.200.0.1: ICMP time stamp query id 42527 seq 0, length 20
21:52:14.049248 IP (tos 0x0, ttl 63, id 0, offset 0, flags [DF], proto TCP (6), length 40)
  10.200.0.1.80 > 10.0.3.73.57705: Flags [R], cksum 0xc84f (correct), seq 3007920764, win 0, length 0
21:52:14.049508 IP (tos 0xc0, ttl 63, id 10678, offset 0, flags [none], proto ICMP (1), length 68)
  10.200.0.1 > 10.0.3.73: ICMP 10.200.0.1 protocol 1 port 19680 unreachable, length 48
  IP (tos 0x0, ttl 40, id 43949, offset 0, flags [none], proto ICMP (1), length 40)
  10.0.3.73 > 10.200.0.1: ICMP time stamp query id 42527 seq 0, length 20
^C
12 packets captured
12 packets received by filter
0 packets dropped by kernel
```

```
(kali@host-3-73)-[~]
$ nmap -sn -n 10.200.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-27 21:52 GMT
Nmap scan report for 10.200.0.1
Host is up (0.00095s latency).
Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

How many different network packets/methods are sent by nmap for host discovery: **4**

To compare to host discovery on a local network, run nmap, to perform only host discovery on a single host again, but this time against the host which was found by your arping scan, which should be in your dump3 output file. In your 2nd terminal window again have tcpdump running to monitor the network traffic generated.

What is the type of packets nmap has used for host discovery on the directly connected network - which protocol?

```
(kali@host-3-73)-[~]
$ sudo tcpdump -ieth0 -nn -vv
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
21:51:58.122514 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.78 tell 10.0.3.73, length 28
21:51:58.122877 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.78 is-at e2:b5:36:6a:c3:e9, length 46
21:51:58.320134 ARP, Ethernet (len 6), IPv4 (len 4), Request who-has 10.0.3.73 tell 10.0.3.78, length 46
21:51:58.320160 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.3.73 is-at 00:0c:49:00:03:90, length 28
21:52:14.047375 IP (tos 0x0, ttl 45, id 2047, offset 0, flags [none], proto ICMP (1), length 28)
```

nmap packet type (use upper case): **ARP**

Now lets run nmap, to perform host discovery on a section of a remote network, and save the output as a grepable data file.

nmap -PE -oG /home/kali/out1 -sn 10.200.0.1-20

This saves the search information as a file called /home/kali/out1 in a "grepable" format. Review the file contents once you have created it, taking note of the traffic and protocols used, and the configuration option -PE for host discovery.

```
(kali@host-3-73)-[~]
$ nmap -PE -oG /home/kali/out1 -sn 10.200.0.1-20
Starting Nmap 7.95 ( https://nmap.org ) at 2026-02-27 21:42 GMT
Nmap scan report for hub1-gw.linuxzoo.net (10.200.0.1)
Host is up (0.0023s latency).
Nmap scan report for hub4-gw.linuxzoo.net (10.200.0.4)
Host is up (0.0022s latency).
Nmap done: 20 IP addresses (2 hosts up) scanned in 0.51 seconds

(kali@host-3-73)-[~]
$ cat out1
# Nmap 7.95 scan initiated Fri Feb 27 21:42:39 2026 as: /usr/lib/nmap/nmap --privileged -PE -oG /home/kali/out1 -sn 10.200.0.1-20
Host: 10.200.0.1 (hub1-gw.linuxzoo.net) Status: Up
Host: 10.200.0.4 (hub4-gw.linuxzoo.net) Status: Up
# Nmap done at Fri Feb 27 21:42:39 2026 -- 20 IP addresses (2 hosts up) scanned in 0.51 seconds
```

How many live hosts has the host discovery scan found:**2**

Which protocol have we configured nmap to use for host discovery? (use upper case): **ICMP**

Check Tests: Ready

Default nmap Host discovery packets/methods	PASS
Local Network Default Protocol	PASS
Live Hosts Found	PASS
-PE Host Discovery Protocol	PASS
/home/kali/out1 looks sensible check1	PASS
/home/kali/out1 looks sensible check2	PASS

Process this output file out1, and generate a file "out2" with only the IP addresses of machines which are up; our target machines.

grep Up /home/kali/out1 | cut -d" " -f 2 > /home/kali/out2

Check Tests: Ready

/home/kali/out1 looks sensible **PASS**
check1

/home/kali/out1 looks sensible **PASS**
check2

/home/kali/out2 seems right **PASS**

Question 10 Target Port scanning - nmap and netcat

Now that our Host Discovery has found some live hosts/target machines, we can focus on the services running on these targets.

While port scanning, always have another terminal window open, and tcpdump running to monitor the scanning packets being created by the tools. Typically tcpdump with -vn is sufficient and add -X if you want to see the content of the packets.

Use nmap to analyse the ports open on 10.200.0.1. As large scale scanning can take a long time to run, restrict your port scan to TCP ports between port numbers 50 to 100 inclusive. List the open port numbers you find with commas between them in the box below (e.g. if ports 50 and 60 are open, the answer would be "50,60"). The numbers in your list must be sorted (smallest number first), and the box is space sensitive so dont have extra spaces!

IMPORTANT. Linuxzoo security may shut you down if you produce too many packets too quickly! Use the following options for nmap to control the scanning traffic generated, or you may be shunned or exited from LinuxZoo. Even with these controlled scanning options, scans can take quite a few seconds to complete.

```
nmap 10.200.0.1 -n -p50-80 --max-retries 3
```

Open ports: **53,80**

Stop the tcpdump monitoring. Review the first 4 packets sent by nmap. Review the help, and identify the flag to stop these from being sent: **-Pn**

Check Tests: Ready

Identify of open ports on **PASS**
10.200.0.1

Identify nmap Flag

PASS

Use a tcpdump filter to only capture traffic to or from port 80.

```
sudo tcpdump -vni eth0 port 80
```

Capture the port 80 traffic generated, while using nmap to carry out a port scan of the target at 10.200.0.1 using a fully connect scan, with -sT. Store this information in /home/kali/nmapfull.

Now you have identified the nmap 'no host discovery' flag -Pn, always use this if the live host is known, to save extra packets being generated.

Check Tests: Ready

nmapfull looks ok PASS

nmap full connect port scanning

Using 'S' for SYN, '.' for ACK, and 'R' for RST (which is how tcpdump shows the "Flags" in the tcpdump), state the handshake involved in the 'full connect' scan captured in the previous question. Separate each packet with a comma. For instance, if the handshake on port 80 was SYN, then ACK, then SYN+ACK, then the answer would be "S,.,S.".

Handshake in nmapfull: **S,S,,,R.**

Check Tests: Ready

nmapfull looks ok PASS

nmapfull analysis correct PASS

Use tcpdump to only capture traffic involving port 80.

```
tcpdump -vni eth0 port 80
```

Capture the port 80 traffic generated, while using nmap to carry out a port scan of 10.200.0.1 using a SYN/half open scan. Store this information in /home/kali/nmapsyn. Remember to continue to use the nmap 'no host discovery' flag!

Check Tests: Ready

nmapsyn looks ok **PASS**

nmap half open port scanning

Using 'S' for SYN, '.' for ACK, and 'R' for RST (which is how tcpdump shows the "Flags" in the tcpdump), state the handshake involved in the full scan captured in the previous question. Separate each packet with a comma. For instance, if the handshake on port 80 was SYN, then ACK, then SYN+ACK, then the answer would be "S,.,S.".

Handshake in nmapsyn: **S,S.,R**

Check Tests: Ready

nmapsyn looks ok **PASS**

nmapsyn analysis correct **PASS**

Continue using tcpdump to capture all traffic using flags -nnv

Carry out a port scan of 10.200.0.1 for the top 30 ports defined in nmap's services file, using a SYN/half open scan. Remember to continue to use the nmap 'no host discovery' flag, and the numeric/no DNS lookup flags!

Consider the first port found using this approach. Review nmap's services file. What is the frequency value of the first port found to be running on the target, which is least often seen running on the Internet (to 3 dec places)**0.048**:

From the nmap services file, the protocol is more frequently run on a different transport protocol. Run an appropriate nmap scan on this port to check if it running on the target. Then run a port scan for the top 5 ports for that transport protocol. What is the service found running with the highest frequency in the nmap-services file:**ipp**

Check Tests: Ready

Identify nmap frquency **PASS**

Identify nmap protocol **PASS**

nmap Port Scanning with Version Fingerprinting

Use nmap, limiting yourself to port 80, and perform application version identification/fingerprinting of the service running on port 80 on 10.200.0.1 (-sV). Enter the version NUMBER returned by nmap, ignoring all other information. So if it returns "Apache httpd 3.0.0 ((Redhat) OpenSSL)" the answer is "3.0.0".

Port 80 version:**2.4.62**

Check Tests: Ready

Version **PASS**
correct

Repeat the above scan, but introduce "--version-trace". This flag means nmap will output detailed trace information about the version fingerprinting/matching it is carrying out. It will highlight the fingerprint it matches on for the each service it identifies, or if it does not have a match it will direct the user to report the new service fingerprint. Look in the nmap trace information for the 'Service Scan Match', where it details the GetRequest fingerprint it has matched on. It should display the line number of the specific service fingerprint it matched on from nmap's services probes file.

Line of nmap-service-probes:**10693**

Check Tests: Ready

Line **PASS**
No

Use "locate" to find the nmap-service-probes definition file, and then do

head -LINE FILE | tail -1

where LINE is the line number from the previous question and FILE is the located filename. So if the file was /var/1 and the line number was 1000, you would do "head -1000 /var/1 | tail -1".

First 2 words of the line:**match http**

Check Tests: Ready

Line	PASS
words	

Use nmap, limiting yourself to port 80, and perform OS fingerprinting of 10.200.0.1. Save the output of running this command to /home/kali/finger. Have a look at the file too, and see what nmap thinks 10.200.0.1 is.

Check Tests: Ready

OS Fingerprint	PASS
test1	
OS Fingerprint	PASS
PASS	

Switch to using netcat, and perform a similar portscan. Review the help for the flags used in our scan below. Note "-z" which means no data is sent. We don't want to communicate with the service yet, and only check the port is open. When we move onto Service Fingerprinting/Version Scanning we will want to send data.

```
nc -vv -n -z -w2 10.200.0.1 50-80
```

Note that there are many "version" of nc, and in some installs you may get an error message with the portscan. In this case switch to "nc.traditional" rather than "nc".

What message do you get for port 50? Connection refused Connection timed out Open Closed Ready Active Firewallled Port not recognised Scan could not be completed due to ARP error

Check Tests: **Connection timed out**

netcat output for port	PASS
50	

Use wireshark or tcpdump, and monitor the netcat scan of the previous question. What sort of scan does this command do by default?

What type of scan? Stealth Inverse Scan ACK Scan Invertive Scan Reflective Scan Half-Open Scan Full-Connect Scan 25%-Open Scan Njerix Scan

Check Tests: **Full-Connection Scan**

netcat output for port 50 PASS